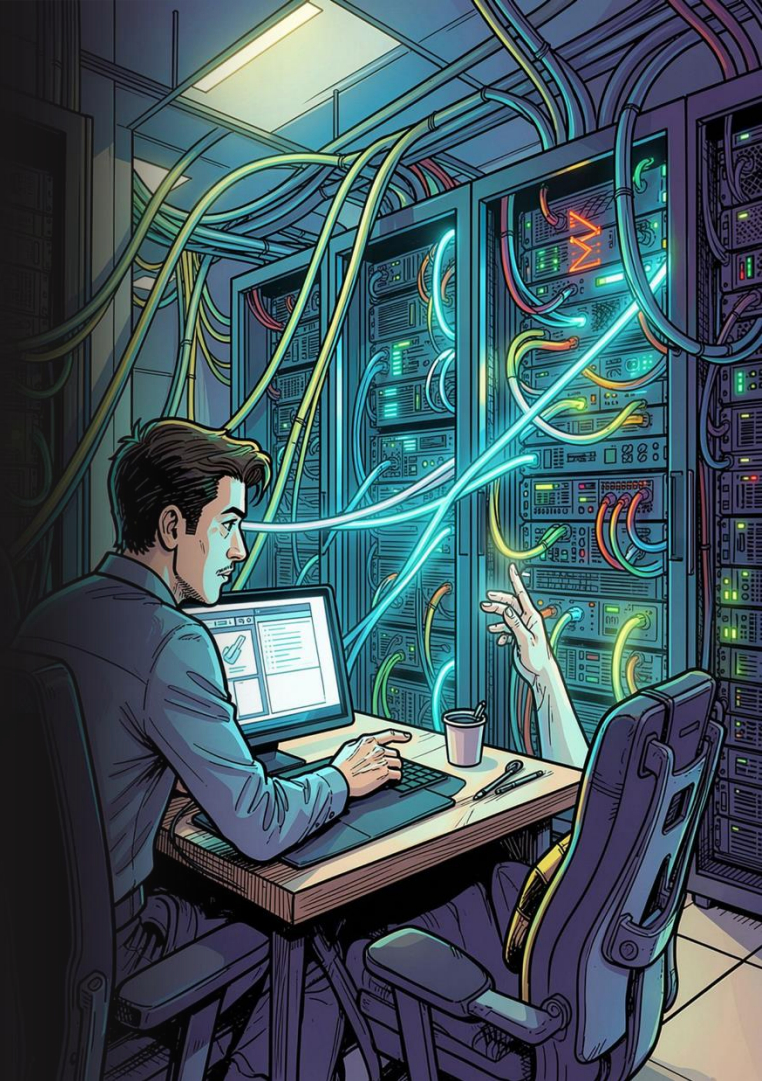


Fundamentos de Python para Backend

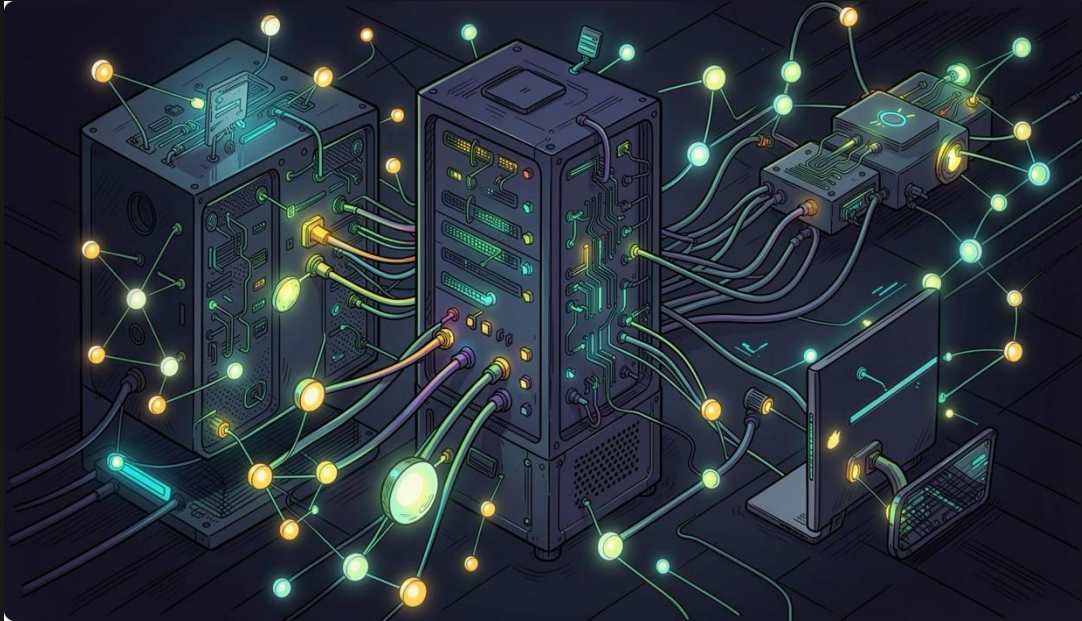
De instrucciones básicas a un programa capaz de procesar solicitudes empresariales.

- Configurar un entorno de desarrollo.
- Representar información mediante tipos de datos.
- Tomar decisiones con estructuras de control.
- Crear funciones y módulos.
- Manejar errores.
- Aplicar buenas prácticas básicas.

❓ ¿Qué tareas realiza el Backend que normalmente no observa el usuario?



El Backend: la lógica detrás de una aplicación



El Backend es el motor invisible de toda aplicación. Se encarga de:

- Recibir y procesar información.
- Aplicar reglas de negocio y validar datos.
- Consultar o almacenar información.
- Gestionar usuarios y permisos.
- Responder a otras aplicaciones.

📄 **Ejemplo:** Al registrar una solicitud de soporte, el Backend verifica los datos, determina la prioridad, registra la solicitud y genera una respuesta.

Preparando nuestro entorno

Herramientas requeridas

- Python 3
- Visual Studio Code o PyCharm
- Terminal o PowerShell
- Entorno virtual (venv)
- Administrador de paquetes pip



El entorno virtual mantiene aisladas las dependencias de cada proyecto.

Creación y activación del proyecto

```
mkdir backend_python
cd backend_python
python -m venv .venv

# Activación en Windows
.\venv\Scripts\activate

# Verificación
python --version
python -m pip --version
```

Estructura inicial

```
backend_python/
├── .venv/
├── main.py
└── requirements.txt
```

De la información empresarial a datos en Python

Cada dato del mundo real tiene un tipo en Python que determina qué operaciones podemos realizar con él.

Información	Tipo	Ejemplo
Código de solicitud	<code>int</code>	<code>1001</code>
Título	<code>str</code>	<code>"Error de acceso"</code>
Tiempo estimado	<code>float</code>	<code>2.5</code>
Solicitud activa	<code>bool</code>	<code>True</code>
Valor inexistente	<code>None</code>	<code>None</code>

Ejemplo en código

```
request_id = 1001
title = "No puedo acceder"
estimated_hours = 2.5
is_active = True
assigned_user = None

# Conversión de tipos
hours_text = "3"
estimated_hours = int(hours_text)
```

❏ ¿Qué tipo devuelve `type("1001")`? ¿Y `type(2.5)`?

Agrupando información relacionada

Lista — elementos ordenados

```
categories = [  
  "Hardware",  
  "Software",  
  "Accesos",  
]
```

Diccionario — claves y valores

```
request = {  
  "id": 1001,  
  "title": "Error de acceso",  
  "priority": "Alta",  
  "active": True,  
}
```

Lista de diccionarios

```
requests = [  
  {"id": 1001, "priority": "Alta"},  
  {"id": 1002, "priority": "Media"},  
]
```



Tomando decisiones y repitiendo procesos

Condicionales `if / elif / else`

```
if priority == "Alta":
    response_time = 2
elif priority == "Media":
    response_time = 8
else:
    response_time = 24
```

Ciclos `for`

```
for request in requests:
    print(request["id"])

for request in requests:
    if request["priority"] == "Alta":
        print(f"Crítica: {request['id']}")
```

`if`

Decide según una condición.

`for`

Recorre una colección elemento a elemento.

`while`

Repite mientras se cumpla una condición.



Predice la salida: ¿Cuántas veces se suma `total` si la lista es `["Alta", "Baja", "Alta"]`? **Respuesta:** 2.

Organizando y reutilizando la lógica

Función con tipo de retorno

```
def calculate_response_time(
    priority: str
) -> int:
    if priority == "Alta":
        return 2
    if priority == "Media":
        return 8
    return 24

hours = calculate_response_time("Alta")
print(hours) # 2
```

Módulo separado

```
backend_python/
├─ main.py
└─ request_utils.py

# request_utils.py
def calculate_response_time(priority):
    # Lógica de cálculo
    pass

# main.py
from request_utils import calculate_response_time
```



Buen criterio: Una función debe tener un propósito definido, recibir solo los datos necesarios y retornar un resultado claro.

Evitar que un error detenga todo el programa

Una excepción representa una situación que impide continuar normalmente. Gestionarla correctamente evita que el programa se detenga de forma inesperada.

Manejo con `try / except`

```
try:
    estimated_hours = int(
        input("Horas estimadas: ")
    )
    print(f"Horas: {estimated_hours}")
except ValueError:
    print("Debe ingresar un número")
```

→ Captura excepciones específicas

Usa `except ValueError`, no `except:` genérico.

→ Muestra mensajes claros

El usuario debe entender qué salió mal y cómo corregirlo.

→ No ocultes errores inesperados

Los errores silenciosos dificultan el diagnóstico.

Generar excepciones con `raise`

```
if estimated_hours <= 0:
    raise ValueError(
        "Las horas deben ser positivas"
    )
```

Reto: procesador de solicitudes de soporte

Construye un programa que reciba una lista de solicitudes y determine su tiempo máximo de respuesta.

Recorrer

Itera sobre todas las solicitudes con un ciclo `for`.

Calcular

Usa una función en un módulo separado para calcular el tiempo.

Gestionar

Maneja prioridades no reconocidas con `try / except`.

Mostrar

Imprime el resultado formateado para cada solicitud.

☐ **Reflexión final:** ¿Qué parte del programa podría reutilizarse dentro de una API? La función que valida la prioridad y calcula el tiempo de respuesta.